FIT3184 2026 S2 Assignment 2 (Group) Marking Rubric

Project: Aussie EcoGuard: Friends & Foes — A Multi-Cloud Serverless Invasive Pests & Pets Monitoring Platform
Total Marks: 100 Marks (Weight: 40%)

Grading Scale

- **100% (Fully Implemented):** The feature is fully functional, robust, and satisfies all specifications without errors.
- **50% (Partially Implemented):** The feature is present but exhibits bugs, misses edge cases, or only partially satisfies the requirements.
- **0% (Not Implemented):** The feature is missing, non-functional, or completely fails during the demonstration.

Detailed Rubric Breakdown

Category & Criterion	Max Mark	100% (Fully Implemented)	50% (Partially Implemented)	0% (Not Implemented)
1. Authentication and Authorisation (10%)				
1.1 Sign-up, Sign-in, Sign-out	**4**	AWS Cognito securely manages registration (capturing email, first name, last name, password), email address verification, temporary password changes, login, and sign-out.	Cognito is configured, but certain features (e.g., email verification flow, mandatory user attributes, or password reset) are buggy or incomplete.	Authentication is missing, broken, or not integrated with AWS Cognito.
1.2 Access Control & Redirection	**3**	Unauthenticated users are strictly blocked from accessing application pages and endpoints (except sign-up/sign-in) and are redirected appropriately.	Partial endpoint protection; some routes or endpoints remain unprotected, or session redirection behaves erratically.	No access protection; pages and API endpoints are publicly accessible without authentication.

1.3 Fine-Grained Permissions & Multi-Cloud Auth (IAM)	**3**	File uploads, serverless invocations, and database access are strictly controlled via least-privilege IAM roles/policies, with secure token forwarding to the secondary cloud provider.	IAM roles are configured but overly permissive (e.g., wildcard access), or cross-cloud authentication token passing is insecure/hardcoded.	IAM policies are missing, non-functional, or credentials are hardcoded into client-side code.
2. Core Functionalities (50%)				
2.1 File & Model Handling (16 marks)				
2.1.1 File Upload & Deduplication	**5**	Files upload reliably via REST API or cloud SDK to object storage (e.g., S3/GCS); duplicate uploads are prevented using file checksum validation.	File upload functions, but duplicate detection fails, throws errors, or relies solely on file names/size rather than content checksums.	Upload fails, or deduplication is not implemented.
2.1.2 Thumbnails & Video Processing	**5**	Event-driven function generates compressed image thumbnails preserving aspect ratio (e.g., OpenCV); videos are processed by extracting **1 frame per 10 seconds**.	Image thumbnails or video extraction functions, but ignores compression/aspect ratio, or extracts frames at an incorrect rate (e.g., every frame or 1 frame/sec).	Thumbnail generation or video processing is absent or completely fails.
2.1.3 ML Tagging, Extensibility & DB Insertion	**6**	Serverless function triggers ML detection (MegaDetector, YOLO, or Rekognition), saves detected	ML detection works but fails to persist complete records (e.g., missing URLs or file types),	ML tagging or database indexing fails or is not implemented.

		species tags, file type, and URLs to the DB; architecture allows model updates without lambda code changes.	requires manual triggers, or is tightly coupled so model versions cannot be decoupled.	
2.2 Queries & Retrieval (24 marks)				
2.2.1 Find by Tags (Counts & Species)	**5**	JSON API processes queries with minimum counts (e.g., {"fox": 2}, {"feral pig": 1, "cane toad": 3}) using logical AND, and queries for specific species, returning correct media URLs.	Query executes but fails logical AND logic (defaults to OR), ignores minimum count thresholds, or returns malformed URLs.	Tag and count queries fail or are not implemented.
2.2.2 Find by Thumbnail URL	**3**	API reliably resolves and returns the corresponding original full-size image URL when provided with a thumbnail URL.	Endpoint functions partially but fails on certain file formats or returns invalid full-size URLs.	Thumbnail-to-image lookup is not implemented or fails.
2.2.3 Find by Uploaded File	**5**	API accepts an uploaded media file, detects species tags on the fly, and queries the DB for matching files; the temporary query file is strictly NOT permanently stored or indexed.	Query detects tags and finds matches, but improperly retains the uploaded query file in cloud storage or writes it to the database.	Upload-based search is non-functional or fails to detect tags.
2.2.4 Lost Pet	**5**	System allows users to search for	Lost pet search is partially	Lost pet search

Search Feature		missing pets by providing a pet image or descriptive tags (e.g., breed, color) against public sightings across the database.	implemented (e.g., works only with text tags but cannot process input images, or searches fail to filter sightings).	functionality is missing or non-operational.
2.2.5 Conversational Chatbot (AWS Lex)	**6**	Integrated AWS Lex chatbot accurately parses natural language queries (intents/entities), invokes backend serverless functions, and presents matching media links to the user.	Chatbot is connected but intent/slot recognition is fragile, fails to trigger backend query lambdas, or returns raw unformatted responses.	Chatbot integration is missing or does not interact with the system.
2.3 Data Management & Notifications (10 marks)				
2.3.1 Manual Bulk Tag Editing	**3**	POST API allows bulk addition (operation: 1) and removal (operation: 0) of tags across multiple URLs; unassigned tags targeted for deletion are safely ignored.	Tag editing works for single files but fails on bulk payloads, or misinterprets the binary operation flag.	Manual tag management is missing or throws server errors.
2.3.2 File & Record Deletion	**3**	API deletes specified files, videos, and associated thumbnails from cloud storage while atomically removing matching database entries.	Storage files are deleted but orphaned database entries remain (or vice versa), or bulk deletion fails.	File deletion is not implemented.

2.3.3 Tag-Based Notifications	**4**	Serverless messaging service (AWS SNS or secondary cloud equivalent) reliably sends email alerts when files containing monitored species/tags are uploaded or updated.	Notifications trigger intermittently, lack tag-filtering logic (sends alerts for all uploads), or contain broken message bodies.	Notification mechanism is not implemented.
3. User Interface (20%)				
3.1 Auth & Upload UI	**5**	Clean web UI supporting user sign-up, sign-in, and file/video uploads, providing clear visual feedback on upload success and deduplication status.	UI works but lacks user feedback/error states, has layout defects, or requires manual page refreshes.	No UI provided for authentication or uploading.
3.2 Query & Lost Pet Search UI	**5**	Intuitive interface for running tag/count queries, uploaded-file searches, and lost pet searches, displaying responsive thumbnail galleries that expand to full-size media.	Query UI functions, but thumbnail previews are missing, clicking thumbnails fails to load full images, or lost pet search UI is disconnected.	No UI provided for queries and search features.
3.3 AWS Lex Chatbot UI	**5**	Interactive, user-friendly chatbot widget embedded in the web application allowing real-time text interactions and clickable media results.	Chatbot UI is present but poorly integrated, crashes on unexpected inputs, or fails to render clickable result links.	No chatbot interface provided in the UI.

3.4 Tag Management & Overall Usability	**5**	Dedicated UI controls for bulk manual tag additions/removals and file deletions; clean, accessible, and intuitive interface design.	Tag editing and deletion UI is clunky, lacks bulk selection capability, or is difficult to navigate.	No UI controls for tag management or file deletion.
4. Demo and Final Reports (20%)				
4.1 Demo Performance (15 marks)				
4.1.1 Multi-Cloud Architecture Presentation	**5**	Clear 3-minute presentation explaining the multi-cloud architecture and trade-offs using official cloud icons; justifies services across >= 2 providers; all members actively participate.	Presentation is rushed, lacks depth on multi-cloud trade-offs, uses unofficial icons, or shows unbalanced member participation.	Presentation not delivered, or demonstrates a lack of understanding of the system's architecture.
4.1.2 Application Demonstration & Q&A	**10**	Live demonstration across all core workflows is smooth and bug-free; team displays comprehensive technical knowledge and answers examiners' questions confidently.	Demonstration has notable execution bugs/delays, or team struggles to answer architectural and code-level questions during Q&A.	Application fails to demonstrate core functionalities during the scheduled interview.
4.2 Final Reports (5 marks)				
4.2.1 Team Report (Max	**3**	Report contains an official multi-cloud architecture diagram, complete contribution table	Missing one or two key elements (e.g., missing user guide, unofficial icons,	Report not submitted or lacks essential sections.

1000 words)		(names, IDs, % contribution, specific roles), testing guide, and private Git repository link with balanced commit histories.	unbalanced Git commit distribution across members).	
4.2.2 Individual Report (Max 500 words)	**2**	Concisely details individual role and technical contributions (~150 words) along with a reflective critique of teamwork, collaboration, and challenges faced (~150 words).	Report is incomplete, lacks substantive self-reflection, or significantly deviates from instructions.	Individual report not submitted.
Total Marks	**100**			

Assessment Policies & Notes

1. **Alternative UI Demonstration Cap:** If a team chooses not to build a web UI, the maximum achievable mark for Section 3 (User Interface) is capped at **14/20 (70% - Distinction)**, provided that all UI interactions are seamlessly demonstrated via alternative mechanisms (e.g., Postman collections, automated CLI scripts, or curl demonstrations). If no UI and no alternative demonstration workflows are presented, Section 3 will be awarded **0 marks**.
2. **Multi-Cloud Requirement:** Solutions must deploy resources across **at least two major cloud providers** (e.g., AWS combined with GCP, Microsoft Azure, or Oracle Cloud). Teams must demonstrate and justify cross-cloud communication during the demo.
3. **Peer Assessment Penalty:** Individual contributions will be verified via the team contribution table, Git commit logs, individual reports, and individual performance during the Zoom Q&A. Non-contributing or deficient members may receive an individual penalty of **up to 50%** deducted from their group mark.
4. **Mandatory Demonstration:** Demonstration via Zoom is compulsory. Failure to attend and demonstrate the working platform results in **0 marks** for the assignment.
5. **Individual Submission:** As this is a group assignment, individual submissions are not allowed; therefore, they will receive 0 marks.